Determinants of Speculative Decoding Efficiency in Resident MoE Inference

Working research manuscript
Author names and affiliations pending

***Abstract*—Speculative decoding trades fewer target iterations for more work per iteration. Prior work establishes how expert activation and batch size affect this tradeoff in mixture-of-experts (MoE) inference. We characterize additional determinants in a resident single-GPU engine: draft availability, actual verification shape, routing dependence, and the service measurement boundary. A BF16 OLMoE/n-gram experiment covers seven batch sizes and eight draft caps with two repetitions per cell. An ordinary-decoding routing model predicts distinct experts in held-out speculative executions with mean absolute error 1.91 using actual verification rows, versus 15.30 using the nominal batch–cap product. Matched single-layer replays support distinct-expert cost fits with held-out mean absolute percentage error of 2.35% in eager mode and 1.92% with CUDA graphs, without establishing a compute transition. Instrumented compiled execution also shows that padding can enlarge physical expert participation beyond valid-row occupancy. Small-batch throughput gains coexist with nearly neutral or negative finite-wave throughput at larger batches, while request-level effective token latency can improve more. Energy and proposal coverage expose further differences between caps. Eager service results retain cold-shape and run-order effects; compiled service performance, neural drafters, and controller benefit remain unmeasured.**

***Index Terms*—Mixture-of-experts, speculative decoding, performance modeling, GPU inference.**

I. Introduction and Scope

Speculative decoding trades fewer target iterations for more work per iteration. In resident mixture-of-experts (MoE) inference, extra verification positions can touch experts that ordinary decoding would not use. MoESD already models the resulting batch-dependent opportunity and demonstrates it experimentally [1]. Our narrower question is: *which observed determinants connect a configured draft cap to expert participation, useful progress, and measured service cost?* A qualitative batch reversal alone is not our proposed novelty.

We characterize actual verification shape, draft availability, graph padding, and service measurement boundaries in one resident single-GPU engine. Expert offloading and inter-GPU communication are outside scope. We use n-gram proposals first; compatible neural sources such as EAGLE-3 require separate target, tokenizer, and interface validation [2], [3]. The target verifier and sampler retain responsibility for acceptance and token emission; $\gamma = 0$ denotes ordinary decoding.

The completed eager campaign covers 56 batch–cap cells and 112 performance records, with separate routing diagnostics and component replay. Two service repetitions per cell do not establish statistical significance. Route-probability fit and test use different speculative caps but shared prompts. Compiled routes are instrumented diagnostics; compiled service performance, neural drafters, and controller benefit remain pending. The implemented controller is a vehicle for testing decision value against strong native schedules, not an established contribution in performance.

A selected-expert union measures participation, not HBM traffic: caches, tiles, representation, and dispatch affect physical reads. Likewise, finite synchronous waves that drain without refilling measure burst throughput rather than steady-state serving capacity. We retain these distinctions when relating occupancy, layer time, request latency, and energy.

II. Model and Predictions

A. Expert occupancy with top-k routing

For layer ℓ, let $R_{\ell,s}$ be the distinct expert IDs selected for valid token slot s, E_ℓ the routed expert count, and $k_\ell = |R_{\ell,s}|$ when top-k is fixed. Define the per-token inclusion probability

$$q_{\ell,e} = \Pr(e \in R_{\ell,s}), \qquad \sum_{e=1}^{E_\ell} q_{\ell,e} = k_\ell. \tag{1}$$

Linearity of expectation yields the exact identity

$$\mathbb{E}[D_\ell(S)] = \sum_e \left[1 - \Pr\left(\bigcap_{s=1}^{S} \{e \notin R_{\ell,s}\}\right)\right]. \tag{2}$$

If token slots are independent and have stationary inclusion probabilities, this becomes

$$\widehat{D}_\ell(S) = \sum_e [1 - (1 - q_{\ell,e})^S]. \tag{3}$$

No independence among the experts selected by one token is required. If a trace instead reports the fraction $p_{\ell,e}$ of all assignments, with $\sum_e p_{\ell,e} = 1$, then $q_{\ell,e} = k_\ell p_{\ell,e}$. Treating top-$k$ as Sk independent draws with replacement gives a different model and can violate the exact $D_\ell(1) = k_\ell$ check.

Let $m_{t,i}$ be available draft length and $k_{t,i} \le \min(m_{t,i}, \gamma_t)$ the scheduled prefix for request i. An unchunked decode-only call has $S_t = \sum_{i=1}^{B_t}(1+k_{t,i})$ valid rows, including subsequently rejected positions. Nominal $B(\gamma+1)$ is attained only with full prefixes. Chunking, mixed prefill/decode, output limits, and padding are recorded separately. Direct route unions measure D_ℓ; uniform routing is only a reference, not the measured predictor.

Graph execution can process $P_t \ge S_t$ model-forward rows, excluding expert GEMM tile padding. We record valid, padding-only, and total expert histograms. Extra physical participation need not yield useful progress (Section III-G); neither union directly measures bytes or padding time.

B. From occupancy to complete iteration cost

Let $w_{\ell,e}$ be expert weight storage bytes in the deployed representation. The logical routed working-set estimate is

$$\widehat{W}_\ell(S) = \sum_e w_{\ell,e}[1-(1-q_{\ell,e})^S]. \tag{4}$$

Shared experts are recorded separately. For a fixed layer, precision, and expert kernel, the proposed component fit is

$$\widehat{T}_{\mathrm{MoE}}(D,S) = \max(aD, bS) + c, \qquad a,b,c \geq 0. \tag{5}$$

Here D, S are actual fixed-input replay counts, fitted separately for eager and CUDA-graph execution. Full-model graph padding additionally requires physical rows and unions. The coefficients are local empirical costs, not bandwidth, FLOP rates, or identified overhead components. Imbalance, alignment, cache state, and launch behavior may require richer terms. The $aD + c$ form suffices in both measured execution modes (Section III-F), establishing neither zero compute cost nor full-model fixed-cost dominance.

Component replay tests measured D, S against layer time; Equation 3 separately tests routes against D. Neither validates complete-step time. Attention depends on context, query count, and backend; shared logical KV does not imply constant physical reads. Complete cost therefore uses a separate table indexed by batch, scheduled cap, production budget, and execution regime. The host-step boundary includes target verification, next-draft production, sampling, scheduling, and control, without subtracting guessed components or adding overlapping phases.

C. Accepted work and the decision rule

Let M be the available draft length before truncation under production budget G. For a chosen cap $\gamma \leq G$, distinguish availability $v_j = \Pr(M \geq j)$ from conditional prefix survival $s_j = \Pr(\text{first } j \text{ accepted} \mid M \geq j)$, for $j \leq \gamma$. These probabilities refer to the same state and budget; their product requires no independence assumption. With one correction/bonus token and no terminal or output-budget censoring, expected emitted tokens per request per round are

$$A(\gamma; G) = 1 + \sum_{j=1}^{\gamma} v_j s_j. \tag{6}$$

Empty proposals contribute no availability but retain lookup/target costs. Availability precedes truncation; ranks beyond G are unobserved. Prefix survival is estimated among requests scheduled to rank j, counting earlier rejection as failure. EOS/output-cap-censored cases are excluded from this uncensored estimator but retained in population reports.

Full availability and rank-invariant conditional acceptance α_g yield $1 + \alpha_g + \cdots + \alpha_g^\gamma$. The code's scalar $\bar{\alpha} = C_a/C_n$ instead pools reached positions: $a = \sum_i a_i$ and $n = \sum_i \min(a_i + 1, k_i)$ for accepted-prefix lengths a_i and scheduled lengths k_i. Positions after first rejection are untested. Counts update as $C_a \leftarrow 0.9C_a + 0.1a$, $C_n \leftarrow 0.9C_n + 0.1n$, with prior $(0.7, 1)$; zero-exposure steps neither update nor decay. This pooled ratio need not equal a rank's conditional probability. The policy primarily uses empirical prefix survival and availability; $\bar{\alpha}$ supplies initial/unseen-rank geometric priors. Prefix/availability EWMAs weight eligible-step proportions; count decay is also per eligible step, not elapsed time. Unseen availability starts at one; priors, rank-population changes, and monotone projection require validation.

For a stable batch and matched ordinary-decoding baseline,

$$\widehat{\mathrm{Speedup}}(B,\gamma) = \frac{\widehat{A}(\gamma; G)\widehat{T}_{\mathrm{round}}(B, 0; 0)}{\widehat{T}_{\mathrm{round}}(B, \gamma; G)}. \tag{7}$$

The online action is selected from a small supported set Γ including zero:

$$\gamma_t^* = \arg\max_{\gamma \in \Gamma} \frac{\widehat{A}_t(\gamma; G)}{\widehat{T}_{\mathrm{round}}(B_t, \gamma; G, L_t)}. \tag{8}$$

The causal policy uses recent availability/survival and offline complete costs, with ties favoring shorter caps. Context can be fixed or binned. Native production uses the selected budget; post-proposal integration holds G fixed and pays its full cost even at $\gamma = 0$. Whether this rule improves on native batch schedules remains unmeasured.

a) Testable scope: Larger AR working sets can reduce relative verification expansion, but compute, attention, and drafting still grow or persist. Occupancy and component timing are separate tests; complete-cost action ranking and service improvement require actual held-out policy execution.

III. Measured Characterization

A. Setting and population

The eager r02 campaign uses OLMoE-1B-7B-0924-Instruct (revision `7f1c97f4`), resident BF16 weights, vLLM 0.26.0, and an NVIDIA RTX PRO 6000 Blackwell Server Edition. The MoE backend is Triton, attention is automatically selected, and KV allocation is 6 GiB. Seven batches $B \in \{1, 2, 4, 8, 16, 32, 64\}$ and caps $\gamma \in \{0, \ldots, 7\}$ yield 56 cells with two runs each. Every run completes the same 64 ShareGPT-derived requests with 128 forced tokens (8,192 total). Routes are captured in separate episodes. GPU lock occupancy is 3,091.24 s including initialization, warmup, and diagnostics.

Only $B = 64$ was warmed; cold-shape and run-order effects remain. Ratios use two-run medians (equal to means here), with raw values retained. Waves drain before new submissions and do not refill vacant slots; windows include submission, prefill, and decode. These are exploratory finite-burst observations, not steady-state capacity or significance estimates.

B. Actual shape predicts participation better than the configured cap

We fit layerwise, token-weighted inclusion probabilities from $\gamma = 0$ across all seven batches and test $\gamma = 1$–7. Decode-only histograms satisfy top-k conservation and include rejected positions; prefill/mixed steps are excluded. The 3,472 fit and 23,568 test layer-steps share prompts and are correlated, not independent system samples.

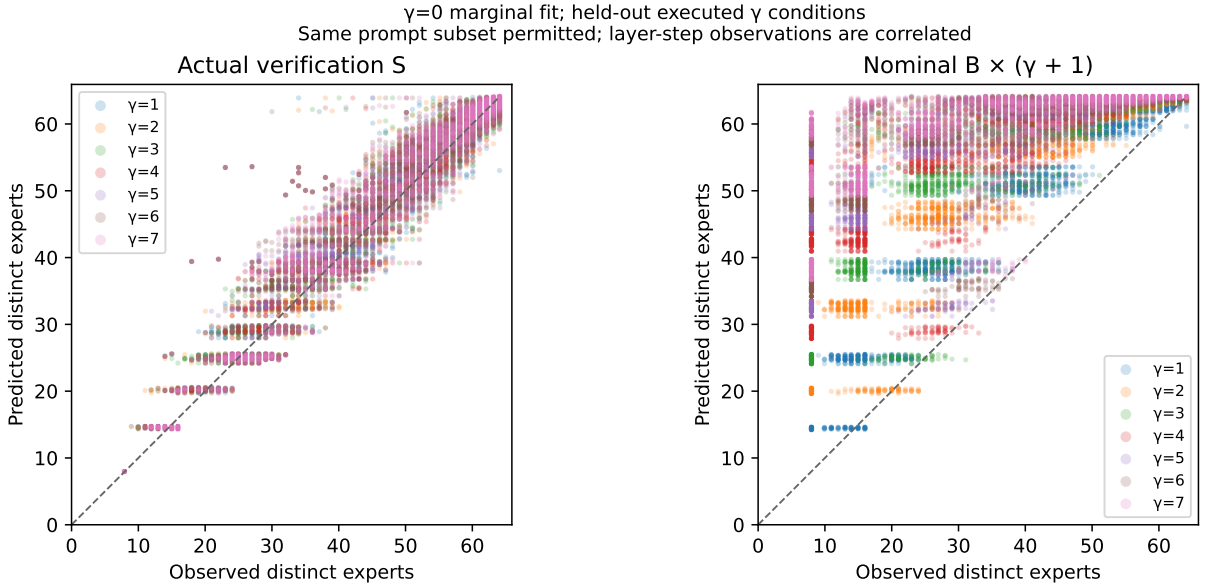


Fig. 1. F1: expert-occupancy prediction using the same AR-fitted inclusion probabilities q_e in both panels. Left uses actual valid verification rows S; right uses nominal $B(\gamma+1)$. Fit: 3,472 AR layer-step observations. Test: 23,568 observations from $\gamma = 1$–7 executions, including rejected draft positions. The prompt subset is shared, so this is held-out execution-condition validation, not independent-workload validation. All eligible decode points, including the first step, remain in both panels. Logical expert counts do not measure HBM traffic or time.

Actual S gives MAE 1.9145 experts and RMSE 3.1453 (Figure 1); nominal $B(\gamma+1)$ raises MAE to 15.2994 with the same q_e. Actual active batch with nominal cap still gives 13.6975, so correcting wave tails alone is insufficient. This isolates shape input, not nonuniform versus uniform routing, and validates neither independent-workload transfer nor action ranking.

a) Structured residuals: First decode steps contribute 784/23,568 observations (3.33%) but 25.78% of squared error: MAE is 5.73 versus 1.78 later. All remain included. AR layer 0 at $B = 32$ also starts at 26 distinct experts versus 61.07 afterward. First decode processes the first generated token, not directly a template token; shared templates or repeated identities are untested explanations, not draft-specific causes.

C. Throughput and request token latency answer different questions

At $B = 1$, all speculative caps improve median wave throughput by 1.129–1.179× over AR; $B = 2, 4$ give 1.061–1.107× and 1.025–1.066× (Figure 2). All $B = 16, 32$ speculative cells are at or below AR in unrounded ratios; $B = 64$ spans 0.995–1.028×. A full-availability, $\alpha_g = 0.7$ arithmetic example assumes conditions absent from this campaign and supplies no calibrated crossover to test. This is not a universal batch reversal or a refutation of differently conditioned models.

Effective TPOT averages each request's first-to-last emitted-chunk interval divided by 127, then takes the run median. It can improve more than complete-wave throughput (Table I), because early finishers leave unfilled slots while the wave waits for its last request.

A receipt decomposition separates mean first receipt, mean post-first span, and the barrier $\max_i(\text{last}_i) - \text{mean}_i(\text{last}_i)$.

TABLE I
DIFFERENT OBJECTIVES ON THE SAME R02 EXECUTIONS. RATIOS COMPARE TWO-RUN MEDIANS WITH MATCHED AR. EFFECTIVE-TPOT SPEEDUP IS AR TIME DIVIDED BY SPECULATIVE TIME; THROUGHPUT SPEEDUP IS SPECULATIVE RATE DIVIDED BY AR RATE.

B	γ	Wave throughput	Effective TPOT
16	6	0.9999×	1.1299×
64	6	1.0234×	1.1582×

Summed over 64-request waves and averaged over runs, $\gamma = 6$ versus AR at $B = 16$ changes these terms by -0.00436, -0.70788, and $+0.71303$ s; at $B = 64$ by $+0.000305$, -0.25573, and $+0.20864$ s. Initial receipt growth thus does not explain the gap. The barrier is not a recoverable-time bound: refill changes execution. Matched queued service and kernel/fixed-cost attribution remain pending.

Raw repetition values remain important: at $B = 1$, AR throughput is $(82.55, 87.75)$ tokens/s versus $(95.69, 96.63)$ for $\gamma = 1$; at $B = 64$, the corresponding pairs are $(4420.34, 4091.67)$ and $(4211.59, 4299.17)$. A cell near unity cannot be interpreted independently of this run variation and incomplete warmup.

D. Draft availability and progress

All performance decode request-steps, including empty proposals, tails, and terminal steps, enter Figure 3. At $B = 1$, nonempty-proposal rates for $\gamma = 1, 4, 7$ are 18.07/14.09/13.37%; mean draft slots are 0.181/0.562/0.928, and emitted tokens are 1.095/1.149/1.158. These are unconditional population rates, not acceptance conditioned on an n-gram hit or the pooled scalar $\bar{\alpha}$.

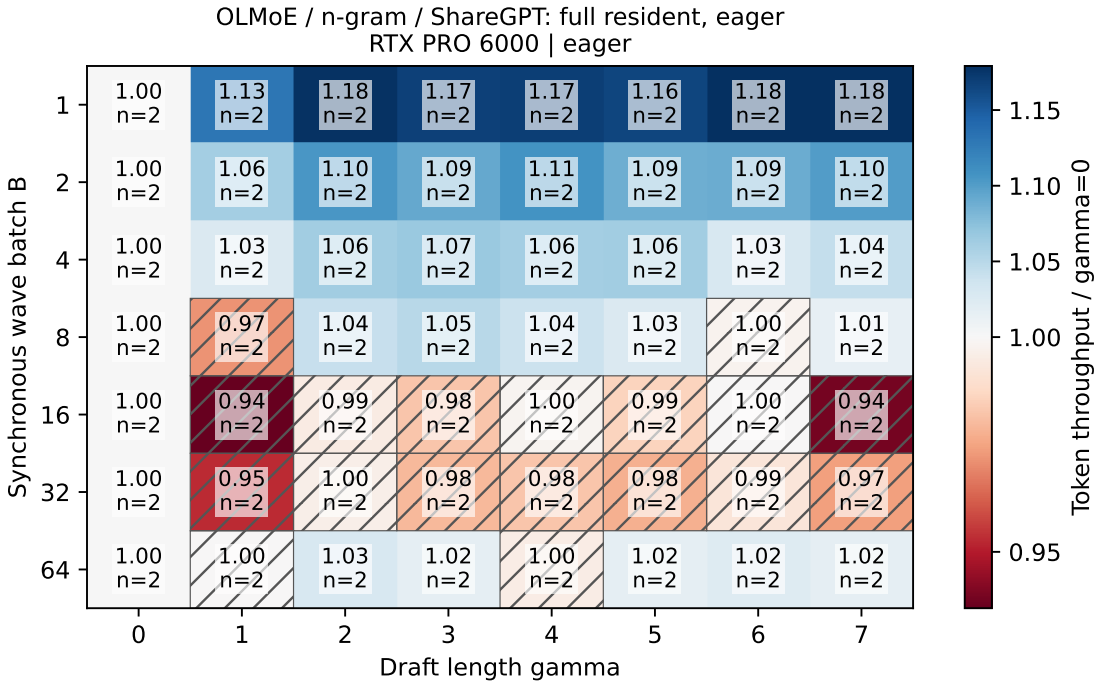


Fig. 2. F2: complete r02 finite-wave throughput surface, normalized to AR at the same initial batch. Each of 56 cells has two repetitions. Windows include prefill and wave drain; hatched cells have an unrounded ratio below one. Cold-shape and order effects remain, so the surface is exploratory rather than a steady-state capacity or significance claim.

Cap alone therefore poorly represents verification work; acceptance without availability cannot predict progress. Lookup, target, and wave costs still determine throughput. Terminal/output limits can make $1+$accepted drafts differ from client emission, so both are retained.

Cold-prior-per-run offline replay changes the $B = 1, \gamma = 7$ scalar estimator's tested-position-weighted pre-step bias from -0.2285 to -0.0137 when replacing EWMA step ratios with EWMA count ratios. Progress MAE stays near 0.282 tokens because availability/prefix weighting is unchanged. Both versions are retained; this is statistical correction on fixed traces, not adaptive execution or performance gain.

E. Energy is a separate measured objective

Energy comes from the cumulative NVML device counter over each measured wave, with no idle-baseline subtraction. At $B = 1$, AR uses 1.4982 J/token; speculative caps use 1.3357–1.3728 J/token (Figure 4). At $B = 64$, AR uses 0.0650 J/token and speculative caps span 0.0638–0.0675 J/token. For example, $\gamma = 6$ at $B = 64$ has throughput ratio 1.0234 but uses 0.0666 J/token. The energy and throughput objectives therefore have different observed orderings; small differences still inherit the two-repeat and warmup limitations.

F. Distinct experts predict a measured layer interval

The eager r03 and CUDA-graph r01 replays use OLMoE layer 0's resident BF16 weights and vLLM `TritonExperts`. All 154 cases match in route histograms, synthetic unit-RMS activations/co-occurrence, seeds, uniform top-k weights, kernel, and tile settings; $S = 1$–148 and $D = 8$–64. Routing quantiles/row extremes select snapshots before timing. Each uses ten warmup calls and five repeats of twenty calls, summarized by the median repeat mean. Both apply 256 MiB unrelated-buffer turnover outside timers; this does not guarantee identical physical cache contents. All graph captures and recorded warmup-output checks pass.

CUDA events cover preparation, alignment, two expert GEMMs, activation, and reduction, excluding routing, attention, drafting, and scheduling. Each mode independently fits 88 snapshots at $B = 1, 4, 16, 64$ and tests 66 at $B = 2, 8, 32$, retaining outliers. Eager $aD + c$ gives $a = 9.471\,\mu$s/expert, $c = 153.261\,\mu$s, MAE 12.731 μs, and MAPE 2.345%. Graph $aD + c$ gives $a = 9.019$, $c = 55.733$ in the same units, MAE 5.868 μs, and MAPE 1.924%. Graph max-form MAPE is 1.920%, with 87/88 fit points and all test points on the D branch (Figure 5). Token-count-plus-intercept/constant MAPE is 21.673/29.517% eager and 28.186/40.041% graph. The tiny max-form increment does not establish a broad compute regime.

Across matched cases, graph/eager event ratios have median 0.7924 and range 0.4899–0.8669; the wall-time ratio median is 0.7943. All 154 graph intervals are lower. Capture and first replay are excluded and recorded separately. These separately timed campaigns are not randomized/interleaved independent

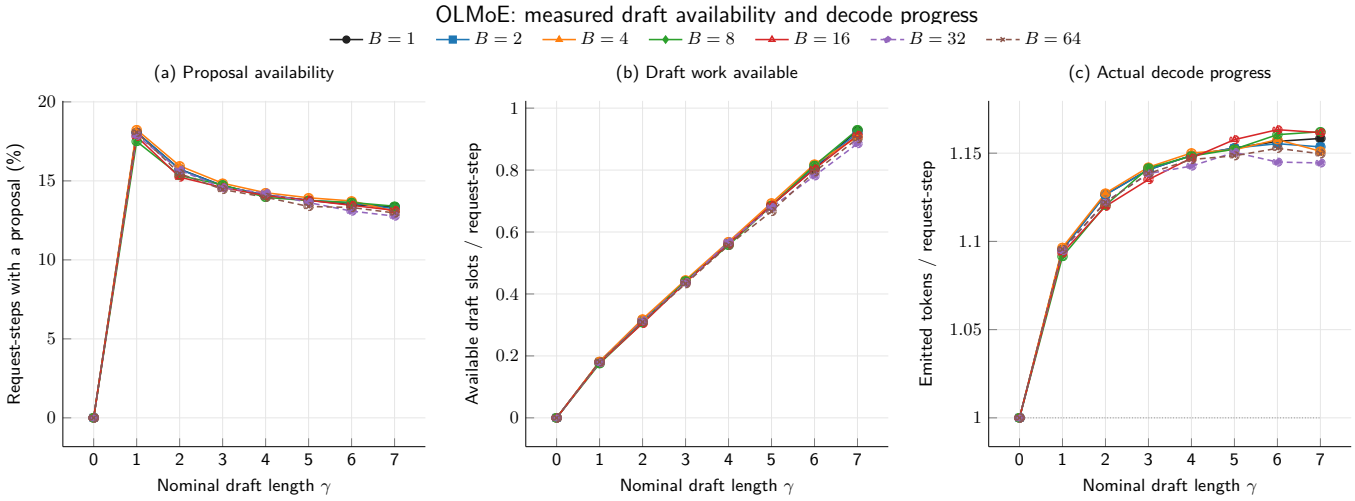


Fig. 3. F5 coverage/progress component: nonempty proposal frequency, mean available draft slots, and actual client-emitted tokens per decode request-step. All 112 performance records contribute, including empty drafts, shrinking drain batches, and terminal steps. Lines summarize two run-level rates and open markers retain their raw values; request-steps are not independent repetitions. This is not a conditional-survival distribution or EWMA-validation plot.

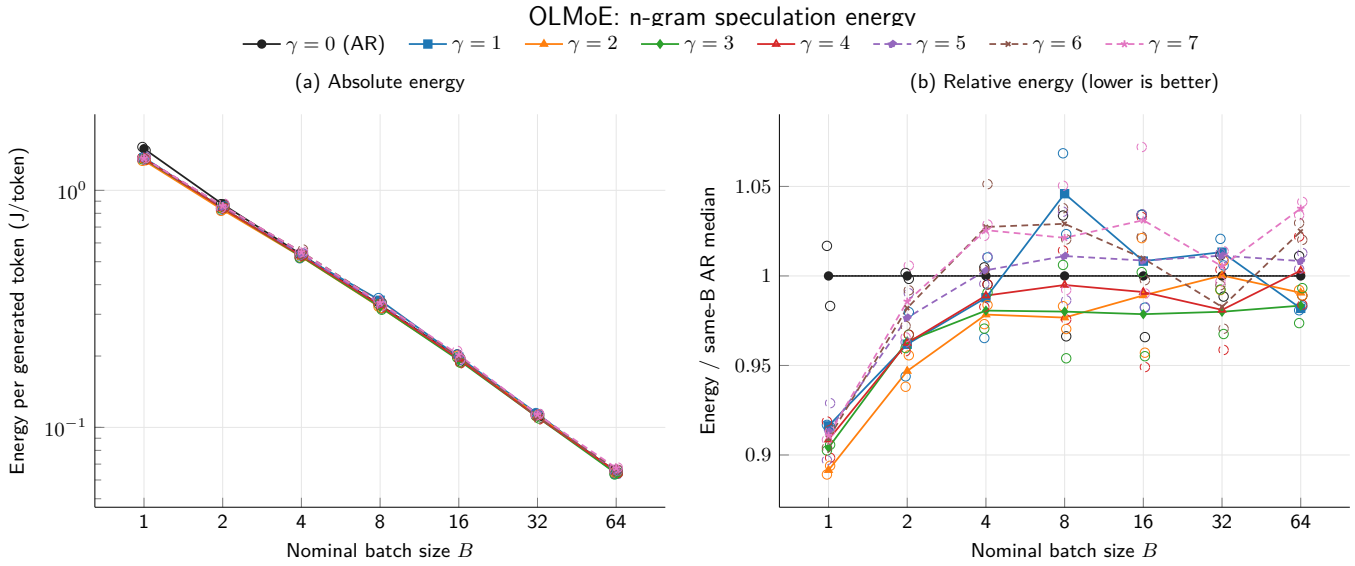


Fig. 4. F4: GPU joules per emitted token over the complete wave windows, including prefill. Lines are two-run medians; open markers are raw runs. Right-panel points divide each run by the same-batch AR median, without paired-run inference, confidence intervals, or denominator uncertainty. NVML device energy is neither whole-system nor idle-subtracted energy; different caps need not optimize throughput and energy simultaneously.

repetitions: the ratios describe an execution-mode contrast, not isolated CPU launch cost or service speedup.

Fixed-layer capture excludes model padding, routing, and attention. Neither fit measures HBM bytes or identifies full-model c_0; the eager token branch is unidentified and graph support is limited to one calibration point. Eager profiling omitted turnover; graph profiling includes it; neither profiler differences nor event-minus-profiler gaps isolate CPU overhead. Complete-step ranking and service benefit remain untested.

G. Physical participation under graph padding

The compiled diagnostic runs $\gamma \in \{0, 4\}$ at $B \in \{1, 8, 32\}$, each with 64 unique requests and 32 forced tokens (Figure 6). All 16 layers and steps pass top-k, histogram, S, and P

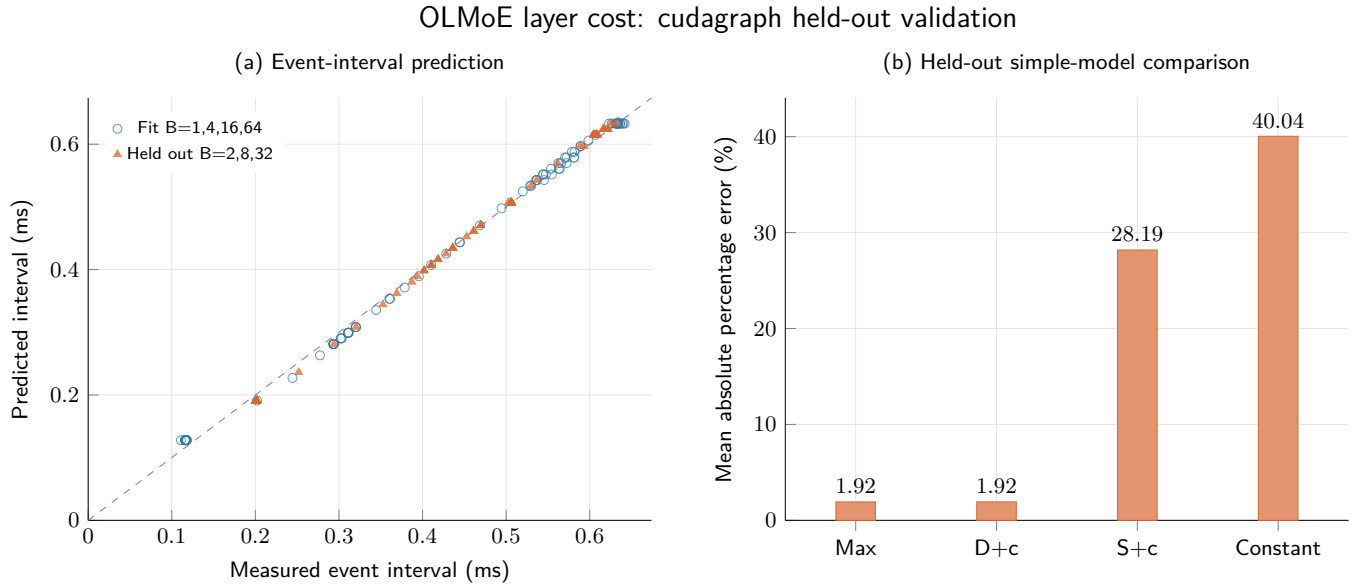


Fig. 5. Single-layer CUDA-graph cost validation with measured D, S: 88 calibration and 66 held-out snapshots use disjoint batches. Points are medians of five within-process repeat means, not independent service runs. The max-form and $aD + c$ have almost equal held-out error; only one calibration point activates the token-count branch.

conservation. AR uses FULL dispatch with $S = P$. At $\gamma = 4, B = 1$, 1,569/1,747 calls use PIECEWISE with $S = 1, P = 5$; the rest use FULL with $S = P = 5$. Mean valid-to-physical D rises 9.62 to 26.73. At $B = 8, 32$, all calls are PIECEWISE; 224/248 and 54/62 are padded, with mean D rising 38.91 to 46.36 and 59.27 to 60.90.

The native V1 graph-size construction rounds capture sizes to multiples of $\gamma + 1$ for full decode graphs; nonuniform calls reuse that table. OLMoE already uses V1, so routed-expert return does not change its size dispatch; the observer reads buffers after the call. The same configuration/shape implies the same padding rule without tracing, but measured unions and times do not transfer directly. The diagnostic scheduler cap is 32 versus planned service cap 64. Instrumentation establishes physical participation, not HBM traffic, padding time, or service benefit.

a) Physical-row prediction: Fitting q_e on compiled AR at $B = 1, 8, 32$ (5,952 tokens/layer) and testing $\gamma = 4$ on shared prompts gives valid-D MAE 0.566/3.438/1.946 from S. For physical D, replacing S with P reduces MAE from 17.675/6.307/1.818 to 3.133/2.898/1.464. These correlated observations hold out execution conditions, not workloads. At $B = 1$, residual bias is $+2.562$ and MAPE 12.864%. All 21,920 adjacent padded layer-step pairs within waves have identical padding histograms. Temporal repetition does not test within-call independence; padding marginals also differ from AR, leaving fresh IID-AR padding unvalidated. Histogram equality proves neither ordered token replay nor cache hits.

IV. Controller and Remaining Evaluation

A. Native controls and complete drafting cost

vLLM 0.26.0 supports inclusive batch intervals mapped to caps, including zero and nonmonotone schedules [4], [5]. This changes the production budget and supplies a strong deployable baseline. The implemented comparison chain has five arms: AR, native static cap, tuned native threshold (including always-on/off), unrestricted native batch table, and the batch/EWMA rule. Cascade is an action-level related-work comparison, not an implemented or promised numerical baseline (Table II). Static caps, threshold plus active cap, and tables are selected on calibration data and frozen, including intermediate-batch rules. A tuned static cap may suffice in stationary states; dynamic-load tests must establish the value of online information.

The current controller truncates a full-budget proposal before scheduler reservation. It saves verification work, not already executed drafting. Costs must distinguish (B, γ, G): native $G = \gamma$ cells cannot calibrate post-truncation $G > \gamma$, including $\gamma = 0$. A truncated threshold is only a cost ablation, not a substitute for the stronger native threshold. Pre-draft selection and neural state continuity require separate validation.

Causal step IDs link proposal, reservation, verification, emitted tokens, rejection, and feedback. Empty drafts, completion, and batch changes are normal transitions. Online EWMA persists across waves, batches, and repetitions within an engine, with recorded initial states; cold-prior offline replay is separate. Target routing and the verifier/sampler remain intact. Correct semantics still require consistent probabilities,

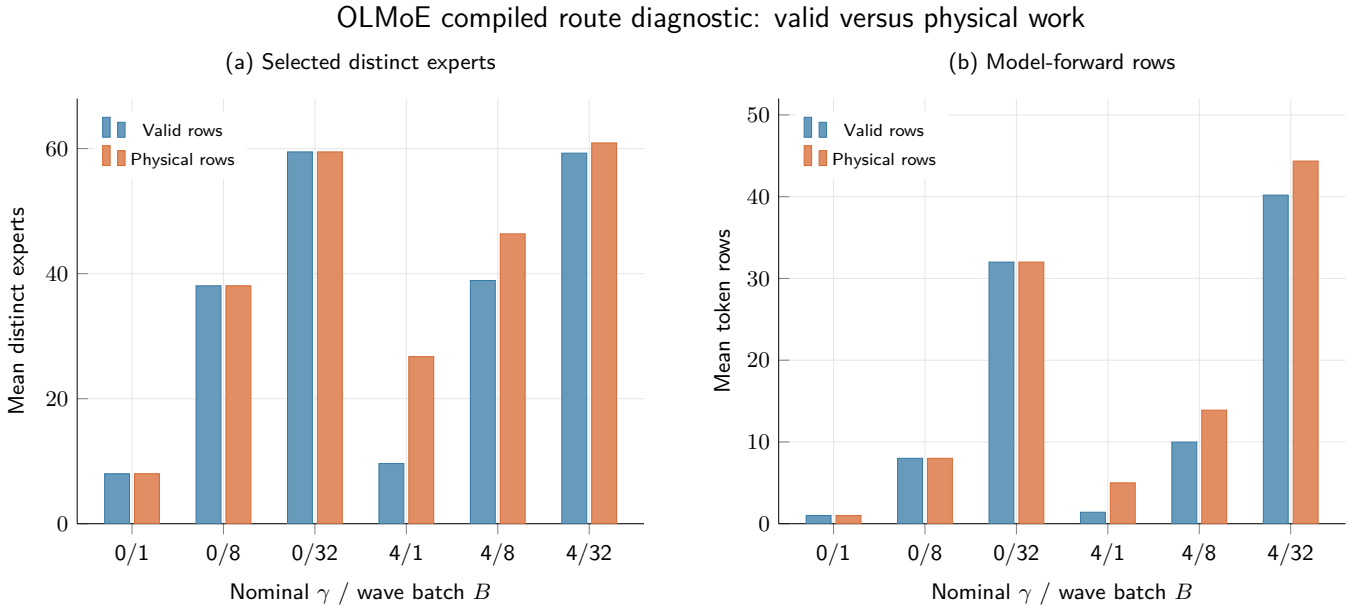


Fig. 6. Instrumented compiled route diagnostic. Physical model-forward rows include graph padding and can activate experts beyond the valid-row union; expert matrix-multiplication tile padding is excluded. Means weight every decode layer-step equally, including shrinking wave tails. The six cases are correlated route observations, without independent confidence intervals or service-time inference.

KV state, stopping, and bookkeeping; integration tests and task spot checks are not distributional proofs.

B. Measurement boundaries and pending evidence

Decision cost includes lookup, updates, IPC, and telemetry synchronization. Detailed route capture is a diagnostic whose online necessity requires ablation. Native r02 timing covers only part of F6: schedule recording consumes 0.1285–0.3628% of host decode time and acceptance updates 0.0356–0.0707%. The control wrapper is nested inside scheduling and cannot be added. These segments neither measure adaptive selection nor total observer cost, and are not subtracted from wall time.

The primary controller objective is effective decode time per emitted token; complete throughput, latency distributions, and device J/token remain co-reported. Run-level repeats retain raw values, warmup coverage, execution mode, run order, failures, and population/window definitions. Shared-process kernel repeats and correlated route events do not increase the independent service sample size. Hindsight best-static cells are labelled diagnostic; splicing favorable steps from different executions is not an achievable causal oracle.

Fixed output length controls neither task quality nor natural work. Separate natural-stop evaluation aligns source IDs and reports EOS, capped, failed, and unobserved outcomes, with fixed first-source inspection samples. Token/prefix agreement is descriptive, not a quality or bug test. NVML cumulative device energy uses the exact wave window; power integration is a separately declared fallback, and startup/compile energy is separate. Baseline and neural-drafter plans share device capacity and a declared 6 GiB KV allocation, not equalized total residency: combined weights, metadata, and achieved KV must be reported.

a) Evidence map: F1 validates eager occupancy and local layer cost on their respective holdouts and diagnoses compiled physical participation. F2/F4 measure static eager wave throughput/energy; F5 measures availability/emission and offline tracking; F6 times native CPU segments only. **F3—executed controller improvement over tuned native static, threshold, and batch-table baselines—is pending.** Compiled service, continuous-load comparison, neural drafters, natural-generation checks, adaptive energy, and total observer/discarded-draft costs are also pending. Current evidence covers one exploratory target/drafter/workload, not a completed evaluation.

Hardware, precision, kernel, context, and workload changes require calibration; instruction-set compatibility does not imply performance transfer. Additional models must test a routing or cost prediction rather than serve as a model-count claim.

V. Related Work

MoESD is the nearest explanation and model baseline [1]. We build on its batch-dependent expert activation account. Its published modeling algorithm fits performance parameters; our inspection did not locate a MoESD online optimal-length controller. That limited observation does not establish absence of any unpublished implementation.

TABLE II

T2: ACTION-LEVEL COMPARISON. PUBLISHED MECHANISMS AND NATIVE CAPABILITIES ARE DISTINGUISHED FROM THE FIVE IMPLEMENTED COMPARISON ARMS; THIS TABLE IS NOT A NUMERICAL REPRODUCTION OF THE CITED SYSTEMS.

Work	Information	Action / execution scope	Drafting-cost boundary
MoESD [1]	Batch, expert activation, acceptance, fitted cost parameters	Performance model and fixed-γ runs; no online optimal-$\gamma(B)$ system located in our reviewed evidence	Fixed-cap drafting and verification; an online selector is not evaluated here.
Cascade [6]	Measured iteration utility / useful-token rate	Online test-and-set, disable/restart, back-off, and hill-climbing; original vLLM 0.6.6 implementation	Utility accounts for draft/target iteration work; online probing also incurs cost. Not reproduced here.
AdaEAGLE [7]	Token embedding and target hidden state; trained predictor	Lightweight MLP predicts next-round draft length	Predictor and actual neural drafting cost; requires predictor training. Not reproduced here.
vLLM 0.26 [4], [5]	Active batch and configured inclusive intervals	Native static cap, threshold, or unrestricted nonmonotone cap table, including zero	Selected cap controls production budget; native execution and scheduling costs remain.
This study	Active batch, observed availability/survival, calibrated complete costs	Prefix truncation after a fixed-budget proposal; target routing and verifier retained	Full G cost is paid even at $\gamma = 0$; adaptive and observer overheads require measurement.

Cascade already implements adaptive MoE speculation in vLLM using utility measured across test-and-set phases, including disabling, back-off, and hill-climbing [6]. Table II contrasts its feedback-driven actions with our explicit batch state and calibrated costs. We do not reproduce Cascade or claim superiority over it; pending controller results are scoped to the four implemented AR/native baselines.

AdaEAGLE predicts draft length with a learned lightweight predictor [7]. SpecDec++ predicts conditional acceptance and stops according to accumulated rejection risk [8]; DISCO uses a shallow classifier for draft continuation [9]. They motivate adaptive length but differ in information and training cost. EAGLE-3 is a draft-generation mechanism, not evidence for the value of our controller [2].

Native vLLM dynamic schedules make batch-only adaptation an existing engine capability [4], [5]. Finally, MoE-Spec changes verification expert budgets and permits a quality tradeoff [10]; our main method keeps target routing intact. The optional expert-dropping extension is not part of the present claims.

VI. CONCLUSION

The measured OLMoE/n-gram setting separates configured cap, useful progress, expert participation, and service cost. Actual rows improve held-out-cap occupancy prediction; compiled padding further expands physical participation. Distinct experts predict sampled eager and graph layer intervals without identifying a broad compute transition or full-model fixed-cost dominance. Wave throughput, effective TPOT, and energy yield different assessments. These exploratory determinants guide compiled-service and broader-draft tests; controller benefit and broader generality remain to be established.

REFERENCES

[1] Z. Huang, L. Zhu, Z. Zhan, T. Hu, W. Mao, X. Yu, Y. Liu, and T. Zhang, "MoESD: Unveil speculative decoding's potential for accelerating sparse MoE," in *Advances in Neural Information Processing Systems*, 2025, arXiv:2505.19645, version 4, revised February 16, 2026. [Online]. Available: https://arxiv.org/html/2505.19645v4

[2] Y. Li, F. Wei, C. Zhang, and H. Zhang, "EAGLE-3: Scaling up inference acceleration of large language models via training-time test," in *Advances in Neural Information Processing Systems*, 2025. [Online]. Available: https://arxiv.org/abs/2503.01840

[3] Safe AI Lab, "EAGLE: Official implementation and checkpoint catalog," https://github.com/SafeAILab/EAGLE, 2026, accessed October 16, 2026; model compatibility must be verified per checkpoint.

[4] vLLM contributors, "Dynamic speculative decoding: Documentation at v0.26.0," https://github.com/vllm-project/vllm/blob/v0.26.0/docs/features/speculative_decoding/dynamic_speculative_decoding.md, 2026, accessed October 16, 2026.

[5] ——, "Speculative configuration and dynamic schedule implementation at v0.26.0," https://github.com/vllm-project/vllm/blob/v0.26.0/vllm/config/speculative.py, 2026, configuration and v1/spec_decode/dynamic/utils.py; accessed October 16, 2026.

[6] A. Saxena, P.-A. Tsai, H. Taneja, A. Jaleel, and M. Qureshi, "Utility-driven speculative decoding for mixture-of-experts," arXiv:2506.20675v1, 2025. [Online]. Available: https://arxiv.org/html/2506.20675v1

[7] S. Zhang, H. Wang, D. Ma, Z. Zhu, L. Chen, K. Lan, and K. Yu, "AdaEAGLE: Optimizing speculative decoding via explicit modeling of adaptive draft structures," arXiv:2412.18910v1, 2024. [Online]. Available: https://arxiv.org/html/2412.18910v1

[8] K. Huang, X. Guo, and M. Wang, "SpecDec++: Boosting speculative decoding via adaptive candidate lengths," in *Conference on Language Modeling*, 2025, arXiv:2405.19715v3. [Online]. Available: https://arxiv.org/html/2405.19715v3

[9] J. Mamou, O. Pereg, D. Korat, M. Berchansky, N. Timor, M. Wasserblat, and R. Schwartz, "Dynamic speculation lookahead accelerates speculative decoding of large language models," in *Proceedings of the 4th NeurIPS Efficient Natural Language and Speech Processing Workshop*, ser. Proceedings of Machine Learning Research, vol. 262, 2024, pp. 456–467. [Online]. Available: https://proceedings.mlr.press/v262/mamou24a.html

[10] B. McDanel, S. Li, S. Surineni, and H. Khaitan, "MoE-Spec: Expert budgeting for efficient speculative decoding," arXiv:2602.16052v1, 2026. [Online]. Available: https://arxiv.org/abs/2602.16052